



INTRODUCCIÓN A LOS CMS HEADLESS CON STRAPI

CARMEN ROA VALDIVIA
ANTONIO GÓMEZ DOMÍNGUEZ

ÍNDICE

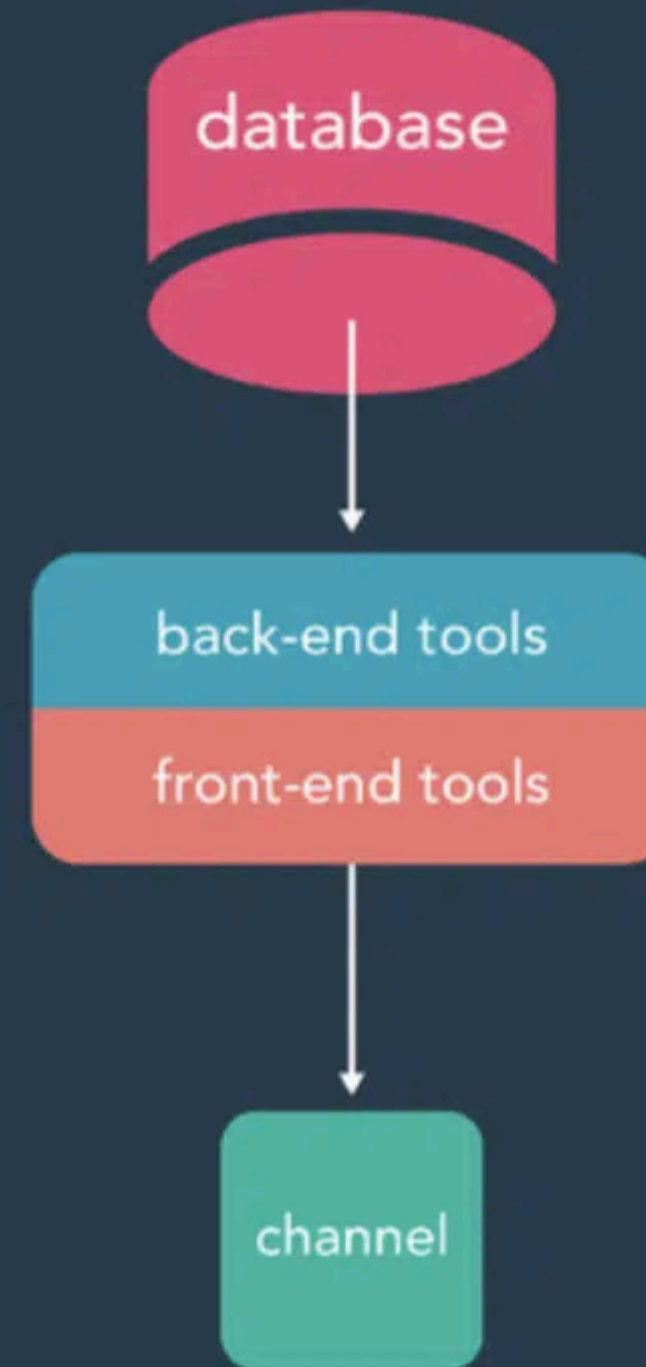
- 01 INTRODUCCIÓN
- 02 CMS HEADLESS
- 03 ¿QUÉ ES STRAPI?
- 04 EJEMPLO APLICADO A
NUESTRO PROYECTO

CMS HEADLESS

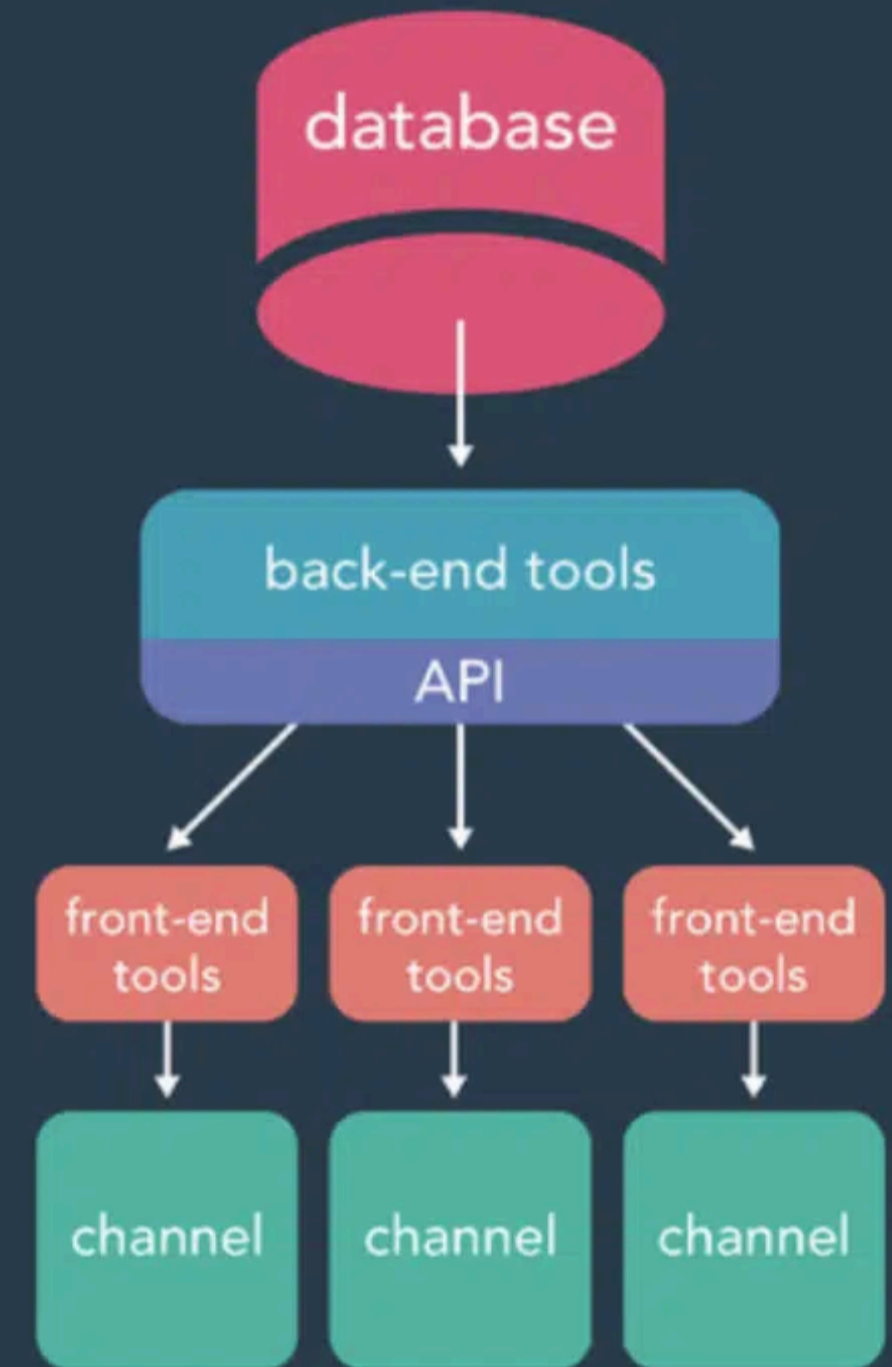
Un **CMS Headless** a diferencia de un CMS tradicional es un sistema de gestión de contenidos que separa el backend (donde se crea y administra el contenido) del frontend (donde se muestra al usuario).

Esto permite entregar el contenido mediante APIs, dándole al desarrollador libertad total para mostrarlo en cualquier dispositivo o tecnología (web, móvil, etc.).

Traditional CMS



Headless CMS



¿QUÉ ES STRAPI?

1

Definición

Strapi es un CMS headless de código abierto, diseñado para facilitar la creación, gestión y distribución de contenido a través de APIs

Está desarrollado en Node.js, un entorno de ejecución que permite ejecutar código JavaScript en el servidor

2

Ventajas del enfoque headless de Strapi

Gracias a su arquitectura headless, Strapi desacopla el backend del frontend, lo que permite consumir el contenido desde cualquier plataforma o dispositivo (web, móvil, IoT, etc.) mediante APIs REST o GraphQL

3

Gestión de contenido flexible e intuitiva

Ofrece una interfaz intuitiva para la gestión de contenido, personalización de modelos de datos y control de roles y permisos, lo que lo convierte en una herramienta flexible y potente para desarrolladores.



EJEMPLO APLICADO A NUESTRO PROYECTO

CONTEXTO DEL PROYECTO

Problema

Necesidad de gestionar contenido (reseñas) sin modificar constantemente el código Java.

Solución

Strapi como CMS headless para:

- Crear/editar reseñas desde un panel admin.

1. CONFIGURACIÓN INICIAL DE STRAPI DESDE TERMINAL

Comando para crear Strapi inicialmente:

```
npx create-strapi@latest mi-strapi --quickstart
```

Inicia Strapi en modo desarrollo

```
npm run develop
```

Después se abre el panel de administración en:

<http://localhost:1337/admin>

```
Project information

Time          Sun May 11 2025 11:52:39 GMT+0200 (hora de vera...
Launched in   5644 ms
Environment    development
Process PID    7072
Version        5.13.0 (node v22.15.0)
Edition        Community
Database       sqlite
Database name  .tmp\data.db

Actions available

Welcome back!
To access the server ⚡, go to:
http://localhost:1337
```

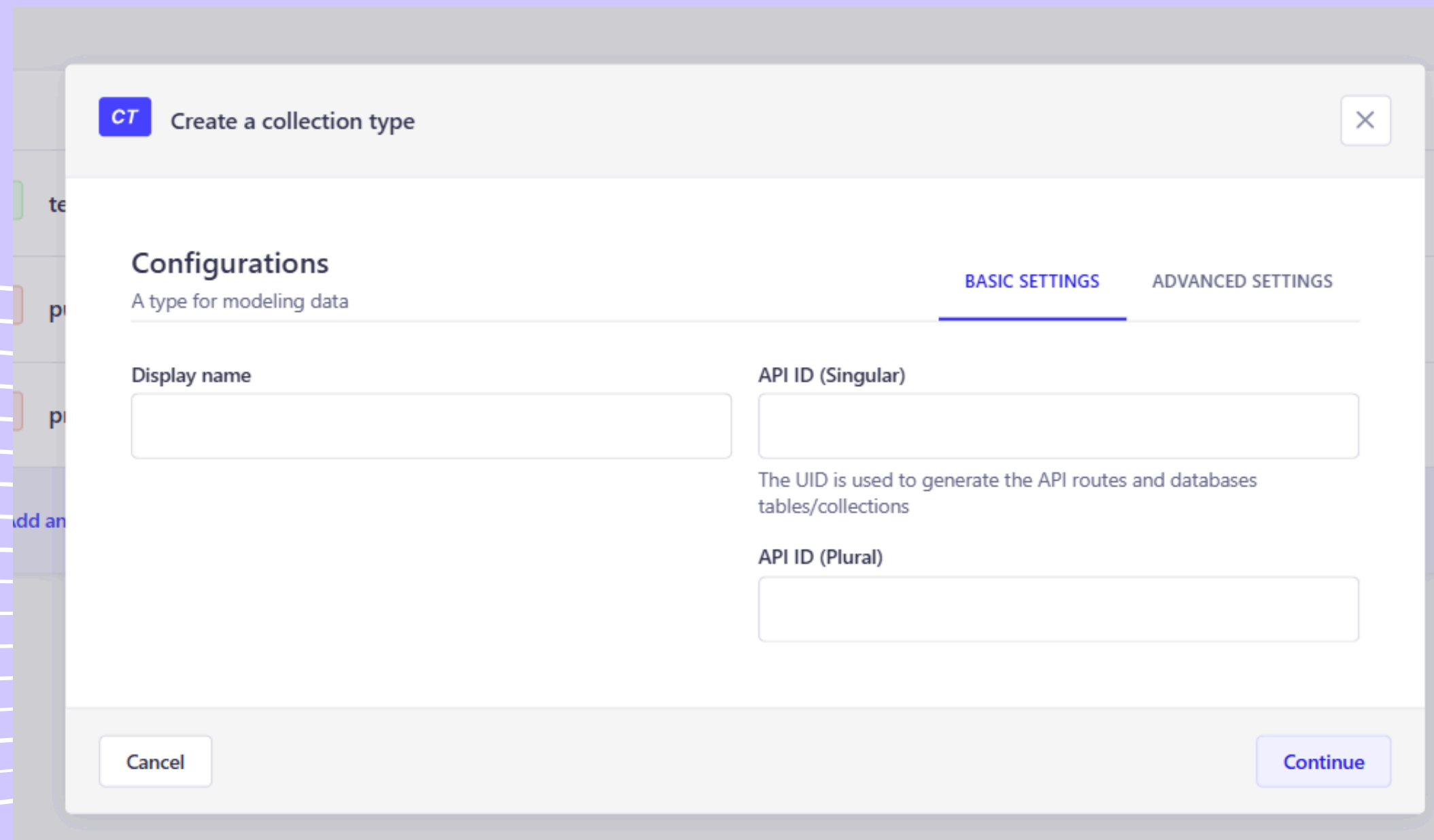
2. CREACIÓN DE LA COLECCIÓN EN STRAPI

1. CREAMOS UN
COLLECTION
TYPE

2. CONFIGURAR
ATRIBUTOS

3. PERMISOS

4. CREAR RESEÑA



The screenshot shows the 'Create a collection type' dialog box in Strapi. The dialog has a title bar with 'CT' and 'Create a collection type', and a close button. The main content area is titled 'Configurations' with the subtitle 'A type for modeling data'. It features two tabs: 'BASIC SETTINGS' (active) and 'ADVANCED SETTINGS'. Under 'BASIC SETTINGS', there are three input fields: 'Display name', 'API ID (Singular)', and 'API ID (Plural)'. A note below the 'API ID (Singular)' field states: 'The UID is used to generate the API routes and databases tables/collections'. At the bottom, there are 'Cancel' and 'Continue' buttons.

CT Create a collection type

Configurations
A type for modeling data

BASIC SETTINGS ADVANCED SETTINGS

Display name

API ID (Singular)

The UID is used to generate the API routes and databases tables/collections

API ID (Plural)

Cancel Continue

2. CREACIÓN DE LA COLECCIÓN EN STRAPI

1. CREAMOS UN
COLLECTION
TYPE

2. CONFIGURAR
ATRIBUTOS

3. PERMISOS

4. CREAR RESEÑA

The screenshot displays the Strapi Content-Type Builder interface. On the left sidebar, the 'Content-Type Builder' section is active, showing a list of collection types: 'Reseña' (selected) and 'User'. Below this, there are sections for 'SINGLE TYPES' and 'COMPONENTS', both currently empty. The main area shows the configuration for the 'Reseña' collection type. At the top, there's a 'Back' link, the title 'Reseña', an 'Edit' button, and buttons for '+ Add another field' and 'Save'. Below this, a table lists the attributes of the collection type:

NAME	TYPE
 texto	Text
 puntuacion	Number
 productID	Number

At the bottom of the table, there is a button '+ Add another field to this collection type'. On the right side of the main area, there is a 'Configure the view' button.

2. CREACIÓN DE LA COLECCIÓN EN STRAPI

1. CREAMOS UN
COLLECTION
TYPE

2. CONFIGURAR
ATRIBUTOS

3. PERMISOS

4. CREAR RESEÑA

← Back

Public

Default role given to unauthenticated user.

✓ Save

Role details

Name*

Public

Description*

Default role given to unauthenticated user.

Permissions

Only actions bound by a route are listed below.

Resena

Define all allowed actions for the api::resena plugin.

RESENA

create

delete

find

findOne

update

Select all

Advanced settings

Select the application's actions or the plugin's actions and click on the cog icon to display the bound route

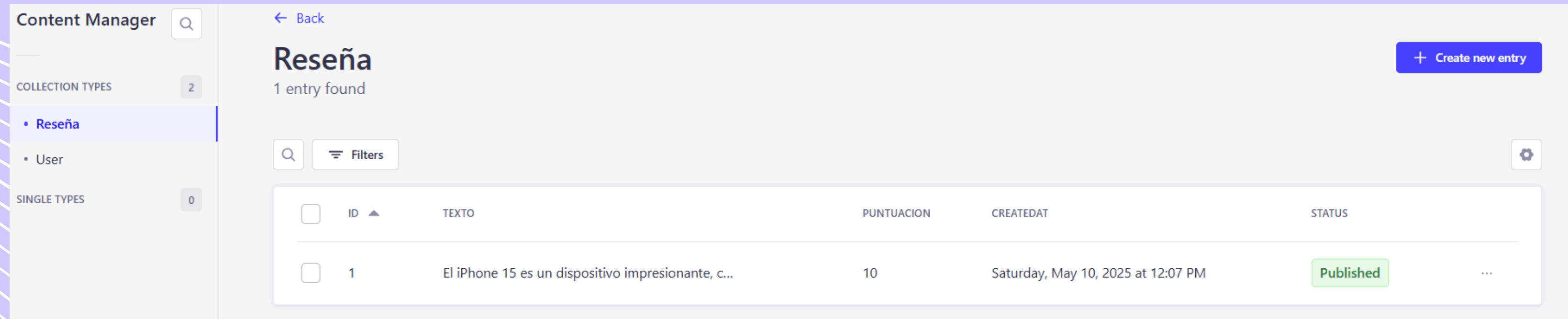
2. CREACIÓN DE LA COLECCIÓN EN STRAPI

1. CREAMOS UN
COLLECTION
TYPE

2. CONFIGURAR
ATRIBUTOS

3. PERMISOS

4. CREAR RESEÑA



The screenshot displays the Strapi Content Manager interface. On the left sidebar, under 'COLLECTION TYPES', the 'Reseña' type is selected, showing a count of 2. Below it, 'SINGLE TYPES' shows 'User' with a count of 0. The main content area is titled 'Reseña' and indicates '1 entry found'. A 'Back' link is at the top left, and a '+ Create new entry' button is at the top right. Below the title, there are search and filter icons. A table lists the entries with columns: ID, TEXTO, PUNTUACION, CREATEDAT, and STATUS. One entry is shown with ID 1, a truncated text description, a rating of 10, a creation date of Saturday, May 10, 2025 at 12:07 PM, and a 'Published' status.

ID	TEXTO	PUNTUACION	CREATEDAT	STATUS
1	El iPhone 15 es un dispositivo impresionante, c...	10	Saturday, May 10, 2025 at 12:07 PM	Published

3. UTILIZACIÓN DESDE JAKARTA EE Y VISUALIZACIÓN EN JSF

01

CREACIÓN DE CLASES

Hemos creado dos clases separadas para facilitar el mantenimiento y así distribuir las responsabilidades:

- ResenaController y StrapiClient

Esta clase es el controlador principal que:

1. Obtiene datos de Strapi a través de StrapiClient.
2. Transforma el JSON de Strapi en una estructura usable en Java.
3. Provee los datos a la vista JSF (resenas.xhtml).

RESENACONTROLLER:

```
@Inject
private StrapiClient strapiClient;

5 usages
private List<Map<String, Object>> resenas;

± Carmen Roa Valdivia
@PostConstruct
public void init() {
    String json = strapiClient.getResenas();
    ObjectMapper mapper = new ObjectMapper();
    try {
        ± Carmen Roa Valdivia
        Map<String, Object> responseMap = mapper.readValue(json, new TypeReference<>() {});
        List<Map<String, Object>> data = (List<Map<String, Object>>) responseMap.get("data");

        resenas = new ArrayList<>();

        if (data != null) {
            // Añadir directamente los elementos de data a la lista de reseñas
            resenas.addAll(data);
            System.out.println("Reseñas cargadas: " + resenas.size());
        }
    } catch (Exception e) {
        resenas = new ArrayList<>();
        System.err.println("Error al procesar reseñas: " + e.getMessage());
        e.printStackTrace();
    }
}

2 usages ± Carmen Roa Valdivia
public List<Map<String, Object>> getResenas() {
    return resenas;
}
```

3. UTILIZACIÓN DESDE JAKARTA EE Y VISUALIZACIÓN EN JSF

01

CREACIÓN DE CLASES

Hemos creado dos clases separadas para facilitar el mantenimiento y así distribuir las responsabilidades:

- ResenaController y StrapiClient

Esta clase es el cliente HTTP especializado que:

1. Realiza peticiones REST a la API de Strapi.
2. Centraliza la configuración de conexiones (URLs, headers).
3. Provee datos crudos en formato JSON para ser procesados por el controlador.

STRAPICLIENT:

```
1 usage  👤 Carmen Roa Valdivia
@ApplicationScoped
public class StrapiClient {

    1 usage
    private static final String STRAPI_API = "http://localhost:1337/api/resenas";

    1 usage  👤 Carmen Roa Valdivia
    public String getResenas() {
        try {
            Client client = ClientBuilder.newClient();
            Response response = client.target(STRAPI_API) WebTarget
                .request(MediaType.APPLICATION_JSON) Builder
                .get();

            if (response.getStatus() == 200) {
                return response.readEntity(String.class);
            } else {
                System.err.println("Error al obtener reseñas. Código: " + response.getStatus());
                return "{\"data\":[]}";
            }
        } catch (Exception e) {
            System.err.println("Excepción al obtener reseñas: " + e.getMessage());
            return "{\"data\":[]}";
        }
    }
}
```


3. UTILIZACIÓN DESDE JAKARTA EE Y VISUALIZACIÓN EN JSF

```
<div class="review-list">
  <h:panelGroup rendered="#{empty resenaController.resenas}">
    <div class="card border-0 shadow-sm p-5 text-center">
      <div class="py-5">
        <div class="mb-4">
          <i class="bi bi-chat-square-text text-muted" style="font-size: 3.5rem;"></i>
        </div>
        <h4 class="fw-light mb-3">No hay reseñas disponibles</h4>
      </div>
    </div>
  </h:panelGroup>

  <ui:repeat value="#{resenaController.resenas}" var="resena">
    <div class="card border-0 shadow-sm mb-4">
      <div class="card-body p-4">
        <div class="d-flex flex-column">
          <p class="mb-3" style="font-size: 1.1rem;">#{resena['texto']}</p>
          <div>
            <span class="badge bg-light text-dark">
              Puntuación: #{resena['puntuacion']}
            </span>
          </div>
        </div>
      </div>
    </div>
  </ui:repeat>
</div>
```

Figura 1. Vista que muestra las reseñas

Figura 2. Visualizacion de la reseña creada en Strapi



GRACIAS